

L'Intelligenza artificiale è una scienza nuova post guerra mondiale (1946)

È suddivisa in sottodiscipline generiche e specifiche.

Essa si occupa di sistematizzare e rendere automatiche attività intellettive.

L'intelligenza artificiale può essere definita secondo il comportamento, il modo di agire, il modo in cui si comporta o razionalmente.

La principale distinzione è tra umanità e razionalità. Turing ha notato che non è importante se un computer sia come un umano, ma che esso sia indistinguibile da esso. LEGGERE TEST TURING

L'IA fa utilizzo delle scienze cognitive, infatti non si può dire che una macchina ^{pense} agisca come un umano se non si è determinato come esso pensa. Se invece si vuole fare in modo che ragioni razionalmente si fa uso della LOGICA. Da un'evoluzione all'utilizzo degli agenti logici sono gli agenti razionali. Essi agiscono nel mondo che ritengono giusto, non è dato che esso sia il migliore, ma per loro lo è.

L'IA è stata possibile grazie a filosofi (che hanno visto le mente come una macchina), matematici (hanno dato gli strumenti per manipolare le logiche), economisti (massimizzare il risultato), psicologi (mente macchina che elabora informazioni), linguisti (linguaggio da usare), ingegneri (programmi e hardware).

La teoria del controllo si occupa della progettazione di dispositivi capaci di agire autonomamente in modo ottimale in base all'ambiente. Questo è stato reso più semplice dall'introduzione del metodo scientifico.

AGENTI

Un agente è qualsiasi cosa che può essere vista come un sistema che percepisce il suo ambiente e agisce su di esso tramite attuatori.

Esso può essere definito come un sistema che prende in input alcune informazioni e produce un output.

Un agente può essere visto come $A = \text{Architettura} + \text{Programma}$.

Il programma agente non farà altro che indicare l'azione da compiere in relazione al suo input o alla ^{sequenza} percezioni. Il programma è concreto, mentre la funzione che indica cosa fare è astratta.

Per capire se un agente agisce nel modo che desideriamo, dobbiamo misurare le sue azioni, viene usata la **PERFORMANCE MEASURES**. E'

intelligente creare la performance measures in relazione agli effetti che vogliamo che l'agente produca sull'ambiente. Essa si basa su una misura, la conoscenza pregressa, le azioni possibili, la seq. prodotta.

Vi possono essere anche azioni mirate a migliorare la azioni future (Int. gathering). Un agente può anche imparare.

Se un agente si basa solo sulle conoscenze pregresse e non dalle percezioni.

AMBIENTI

Gli ambienti rappresentano il problema che gli agenti devono cercare di risolvere. PEAS (task environment) è il primo passo da fare per definire un agente.

PEAS (Performance, environment, actuators, sensors)

Un ambiente può essere:

- osservabile o parzialmente osservabile } certo o incerto
- deterministico o stocastico
- episodico o sequenziale
- statico o dinamico
- discreto o continuo
- singolo o multiagente - cooperative
- competitive
- conosciuto o sconosciuto.

Parz. osservabile, stocastico, sequenziale, dinamico, continuo, multi

Visione a 4 tipi di programmi agente:

- Agenti reattivi semplici: ricevono input e producono output. If else
- Agenti basati su modelli: bisogna tenere memoria dello stato dell'ambiente ed aggiornarlo ogni volta. Bisogna sapere come evolve questo mondo e gli effetti che le azioni hanno su di esso. MODELLO
- Agenti basati su obiettivi: sceglie l'azione che più lo avvicina all'obiettivo. Sfrutta la RICERCA e la PIANIFICAZIONE
- Agenti basati sulle abilità: sceglie l'azione che produce più 'abilità'. Utile quando ci sono più obiettivi, non ottenibili insieme o inanti.
- Agenti capaci di apprendere: learning element, performance element, critic, Problem generator.

Tut: gli agenti possono implementare l'apprendimento.

RAPPRESENTAZIONE TRA STATI

Atomic: ogni stato è indivisibile, ~~è come una~~ ^{considero} black box, ~~considero~~
come un singolo atomo di conoscenza

Caratteristica: Ogni Δ consiste in un insieme fissato di Δ -6 Δ , condivisi
bili da alcuni Δ -essenziali.

Struttura: Ogni Stato contiene più oggetti che hanno le loro

simplest $\rightarrow$... complex

Atomic $\rightarrow$ factored $\rightarrow$ structured.

powerless ~ ~ ~ ~ ~ ~ ~ ~ powerful



formulazione del problema
Gli obiettivi sono fondamentali per limitare e organizzare la ricerca. La definizione d'essi e delle performance measures sono alla base della soluzione. L'obiettivo è un insieme di stati del mondo in cui lo stesso è soddisfatto. Se si conoscono si possono effettuare Ricerche, ovvero sequenze di azioni che portano ad uno d'essi. Dunque FORMULAZIONE $\rightarrow$ Ricerca $\rightarrow$ Esecuzione

Un problema è diviso formalmente in: stato iniziale, possibili azioni, test obiettivo, costo di cammino.

Per rendere più semplice la ricerca durante la formulazione si astrae, ovvero si eliminano tutte le informazioni non necessarie, non invalidando il problema stesso.

La ricerca in seguito può avvenire attraverso un albero o un graph di ricerca, tramite generazione ed espansione. L'output di una ricerca viene valutato secondo 4 parametri: completezza, ottimalità, complessità ^{temp}, ^{spaz}.

b $\rightarrow$ fattore di ramificazione (numero massimo dei nodi figli)

d $\rightarrow$ massima profondità obiettivo più vicino

m $\rightarrow$ lunghezza massima dei cammini

costo ricerca $\rightarrow$ complessità temporale e spazio

costo totale $\rightarrow$ costo ricerca + costo soluzione trovata

La ricerca avviene tramite algoritmi, i quali possono essere informati o non informati.

NON INFORMATI

BREADTH-FIRST SEARCH = completa $O(b^{d+1})$

UNIFORM-COST SEARCH = ottimo ^{più tempo} $O(b^{2+1} \cdot c^{1/2})$

DEPTH-FIRST-SEARCH = $O(b_m)$

DEPTH-FIRST-LIMIT-SEARCH = $O(bL)$.

ITERATIVE DEEPING SEARCH = incrementa LIMIT.

BIDIRECTIONAL = due ricerche in parallelo $O(b^{d/2}) + O(b^{d/2})$

PROBLEMI SENZA SENSORI = non riesce a capire in che stato si trova

CONTINGENZA = pianificazione, ricerca di un dato.

ESPLORAZIONE = non si conosce nulla e bisogna scoprire

INFORMED. $\rightarrow$ heuristic

Per rendere più efficienti gli algoritmi si usa l'euristica, ovvero una funzione che tramite una valutazione ci indica quale stato è migliore, più promettente valutando la sua distanza da un obiettivo. ~~$f(n) = g(n) + h(n)$~~
BEST-FIRST GREEDY $= f(n) = h(n)$. non ottima, non completa, $O(b^n)$

A*

$f(n) = g(n) + h(n)$. E' ottima e completa se $h(n)$ è ammissibile e consistente ^{solo grazie}
ovvero se il costo da n non sia superiore al costo di $n+1 + h(n+1)$.

IDA*

E' un'evoluzione dell'Iterative DS $\rightarrow$ A*. Il valore di taglio è il costo min dei nodi che hanno superato il valore di taglio.

BFS

E' una ricerca BFS con spazio lineare, ovvero tiene in memoria solo il cammino corrente più la migliore alternativa

MA*

Funzione come A*. espande sempre il nodo più promettente cancellandone uno solamente quando la memoria è piena

MA* è linite di memoria

Tutti gli algoritmi possono implementare l'apprendimento $\rightarrow$ tre test di indole.

Per trovare l'euristica il modo migliore è trovare il problema relativo. La soluzione ottima di un rilassato è infatti una buona euristica per il problema originale.

LOCAL SEARCH $\rightarrow$ solo parenti, aggiunge nodi solo come

DESCENT (scendere) $\rightarrow$ Azione, rilassamento, funzione costo, test

AGENTI

Un agente è qualsiasi cosa che può essere vista come un sistema che percepisce il suo ambiente e agisce su di esso tramite attuatori.

Esso può essere definito come un sistema che prende in input alcune informazioni e produce un output.

Un agente può essere visto come $A = \text{Architettura} + \text{Programma}$.

Il programma agente non farà altro che indicare l'azione da compiere in relazione al suo input o alla ^{sequenza} percezioni. Il programma è concreto, mentre la funzione che indica cosa fare è *abstract*.

Per capire se un agente agisce nel modo che desideriamo, dobbiamo misurare le sue azioni, viene usata la **PERFORMANCE MEASURES**. E'

intelligente creare la performance measures in relazione agli effetti che vogliamo che l'agente produca sull'ambiente. Essa si basa su una misura, la conoscenza pregressa, le azioni possibili, la seq. percet.

Vi possono essere anche azioni mirate a migliorare la azioni future (Inf. gathering) Un agente può anche imparare.

Se un agente si basa solo sulle conoscenze pregresse e non dalle percezioni.

AMBIENTI

Gli ambienti rappresentano il problema che gli agenti devono cercare di risolvere. PEAS (task environment) è il primo passo da fare per definire un agente.

PEAS (Performance, environment, actuators, sensors)

Un ambiente può essere:

- osservabile o parz. osservabile
- Deterministico o stocastico
- episodico o sequenziale
- statico o dinamico
- discreto o continuo
- singolo o multiagente - cooperative
- competitive
- conosciuto o sconosciuto.

Parz. osservabile, stocastico, sequenziale, dinamico, continuo, multi

Vi sono 4 tipi di programmi agente:

- Agenti reattivi semplici: ricevono input e producono output. If else
- Agenti basati su modelli: bisogna tenere memoria dello stato dell'ambiente ed aggiornarlo ogni volta. Bisogna sapere come evolve questo mondo e gli effetti che le azioni hanno su di esso. MODELLO
- Agenti basati su obiettivi: sceglie l'azione che più lo avvicina all'obiettivo. Sfrutta la RICERCA e la PIANIFICAZIONE
- Agenti basati sulle abilità: sceglie l'azione che produce più 'abilità'. Utile quando ci sono più obiettivi, non ottenibili insieme o incerti.
- Agenti capaci di apprendere: learning element, performance element, critic, problem generator.

Tutti gli agenti possono implementare l'apprendimento.

RAPPRESENTAZIONE TRASTATI

Atomic: ogni stato è indivisibile, ^{considero} come una black box, ^{considero} come un singolo atomo di conoscenza

Factored: Ogni stato consiste in un insieme fissato di attributi, condivisi tra alcuni di essi.

Structured: Ogni stato contiene più oggetti che hanno relazioni

simplest complex

Atomic → Factored → Structured.

powerless powerful



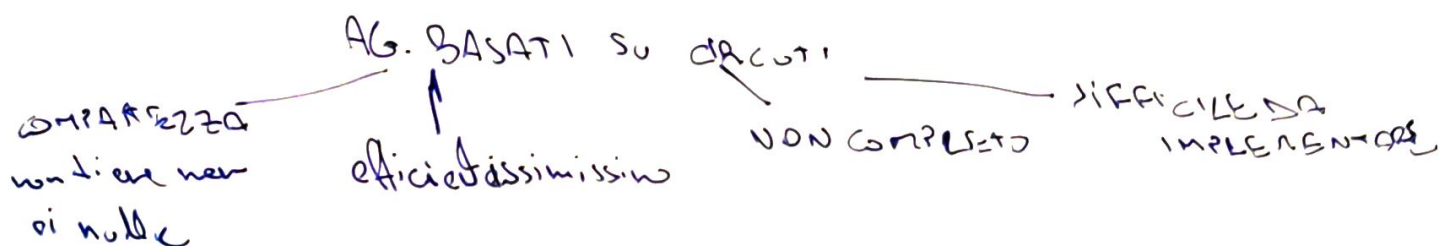
Gli agenti logici sono agenti basati sulla conoscenza, sono flessibili in quanto sono capaci di dedurre stati nascosti del mondo tramite la conoscenza iniziale e le percezioni.

L'elemento più importante per un ag. logico è la KB. È costituita da un insieme di formule logiche che prevedono meccanismi di aggiunta e richieste di informazione, tramite i quali inferisce nuova conoscenza. Le soluzioni di un programma agente è il seguente:

- 1- input di percezione
- 2- KB conoscenza iniziale
- 3- ogni qual volta chiamato in causa l'agente comunica le sue percezioni alla KB e chiede le azioni da eseguire.
- 4- registrare l'azione che esegue.

Questo fa sì che una volta dato delle capacità di apprendere, l'agente diventi completamente autonomo.

Per far sì che questo possa essere gestito l'agente deve rispettare dei concetti fondamentali: SINTASSI, SEMANTICA, MODELLI, INFERENCE
GROUNDING = legame tra ^{logica} agente e mondo reale (diverso da altri)



La pianificazione si occupa di trovare una sequenza di azioni per raggiungere l'obiettivo, essa è detta classica se siamo in un ambiente completamente osservabile, non classica altrimenti. Un agente pianificatore ha una rappresentazione esplicita dell'obiettivo, questo fa in modo che esso sia indipendente in quanto l'obiettivo, (l'espresso della congiunzione di sub-obiettivi) è solo una conj. logica. Esso inoltre lavora indipendentemente su ogni sub-obiettivo.

Il problema fondamentale è quello di trovare un linguaggio capace di esprimere molti problemi.

STRIPS

- Il mondo viene scomposto in condizioni logiche GOALS da conseguire.
- Viene logicamente rappresentato da cond. logiche GOALS l'obiettivo.
- Vengono rappresentate le azioni. Ogni ha delle pre-condizioni e degli effetti.
- Un azione è formata da: NOME AZIONE, PARAMETRI, PRECOND, EFFETTI.
- Una Soluzione non è altro che una sequenza d'azioni che porta ad un obiettivo.

A causa dell'evoluzione STRIPS non è più sufficientemente espressivo, dunque è stato ampliato.

ADL

- Vi sono positivi e negativi
- Il mondo è aperto
- Vi sono i quantificatori $\exists, \forall$
- Vi sono i when e, l'uguaglianza e i tipi

T.A.: i linguaggi sono stati sistematizzati in uno standard detto PDDL

PDDL ha 3 problemi fondamentali: qualificazione, rimozione, frame problem.

PDDL è composto da due file: Dominio, Problemi file.

Vi sono gli oggetti, predicati, stato iniziale, obiettivi, azioni



PIANIFICAZIONE CON RICERCA

Può essere fatta sia per progressione (in avanti) o regressione (in dietro)
Nel primo caso si inizia dallo stato iniziale e tramite azioni si raggiunge l'obiettivo.
Nel secondo si parte dalla soluzione e si cercano gli stati iniziali.

Mentre il primo è completo il secondo lo è parzialmente.
Trovare un'esistente per entrambi non è facile si ricorre dunque al
RELASSATO o al DIVIDE ET IMPERA (ottimiste o negative)

ORDINAMENTO PARZIALE PIU' FLESSIBILE

Non viene specificato quale azione compiere prima, questo ha lo svan-
taggio di non poter determinare quanto ci siano arr. all'obiettivo

La comunicazione è lo scambio di informazioni fra agenti in un ambiente.

~~Durante~~ Un messaggio viene inviato da uno speaker ad un hearer. ~~Durante~~
~~ci si~~ ~~essendo~~ ~~utero~~, l'unità di discorso, espressione di ~~l'azione~~.

Un messaggio può avere vari obiettivi: informare, domandare, rispondere, ecc.

Vi sono due fasi: - quella in cui lo speaker invia $Intention \rightarrow Content \rightarrow Style$
 $P \rightarrow W$

- quella in cui l'hearer riceve $Perc \rightarrow Analysis \rightarrow Disambiguation \rightarrow Interpretation$
 $W \rightarrow W^? \rightarrow P^?$

L'obiettivo è di $P \leq P^?$

Per inviare un messaggio si possono usare 2 modelli:

- Tell and Ask e Formal language

In genere hanno la sintassi, la semantica, pragmatic (insieme sono la semiotica).

Un linguaggio formale ha in sé la grammatica, ~~che~~ che specifica la struttura e la composizione dei messaggi. Un lang formale d'invio distingue e simboli terminali, ogni stringa può essere analizzata e generata da una gram.

Vi sono vari tipi di grammatiche: REGULAR, CONTEXT-FREE, CONTEXT-SENSITIVE, RECURSIVE ENUMERABLE.

La semiotica forma l'Al. linguistica. Per ognuno d'esso distinguono ~~Precondition~~,
~~Act-list~~ ~~delete-list~~ locuzione (preferibili affermative), Illocutione (scopo dell'enunciato),
e Perlocution (conseguenza dell'enunciato). Vi sono vari tipi, rappresentativi, direttivi,
Espressivi, dichiarativi, ecc. Ha due tipi di componenti: PERFORMATIVE $\rightarrow$ lo scopo
PROPOSITIONAL $\rightarrow$ contenuto

La sua semantica può essere descrittiva:

PRE-CONDITION: cosa deve essere valido prima dell'Al

ASS-LIST : cosa deve essere valido dopo

DELETE-LIST: cosa non è più valido dopo

L'Agent Communication Language fornisce agenti: strumenti per scambiare informazioni e conoscenza. L'entità base di un ACL è il messaggio (speech act), che viene inviato tramite protocolli TCP/SMTP. Un ACL descrivono lo stato residente in un linguaggio dichiarativo. L'ACL gestisce proposizioni, regole e azioni con la semantica di esso delegata, il tipo di messaggio, impone il BDI state, ovvero lo stato mentale degli agenti. Vi ne sono due tipi: KQML e FIPA ACL. KQML si basa sul modo dei sistemi software con KB virtuali, vi è una relazione ~ 3 tra KB, una proposizione in notazione formale, un insieme di

KQML è un protocollo ed un linguaggio per scambio d'informazioni tra VKB, indipendentemente dalle sintassi e dall'ontologie. Un agente è visto come una base di conoscenza, dotata di alcune informazioni memorizzate (beliefs) e un obbiettivo (goal). KQML ha 3 livelli: -CONTENT LAYER: il messaggio espresso nella lingua dell'utente
-COMMUNICATION LAYER: edifica delle funzionalità nel messaggio che descrivono i parametri
-MESSAGE LAYER: codifica il messaggio da trasmettere

La KQML semantics è composta da Pre(Agent) condizioni da rispettare prima di inviare, Post(Agent), condizioni che devono essere valide dopo il messaggio e COMPLETION, condizione dello stato finale

KQML fornisce primitive di facilitazioni da supportare le operazioni delle informazioni.

FIPA si propone lo scopo di creare specifici standard che massimizzano l'interoperabilità tra gli agenti. ~~Non~~ Non

VBDFRL S LIDEL

5) Si tratta di canali uguali, semanticamente diversi, non obbligati di lingua, FIPA è più potente nelle operazioni di perfezionare, FIPA non ha fattori

Gli ambienti multi agenti competitivi sono problemi nelle ricerche, in quanto le azioni degli avversari non sono sempre prevedibili e danno origine dunque a casualità.

Questi problemi sono detti giochi.

Nell'IA i giochi appartengono alla categoria dei giochi a somma zero se si tratta di due giocatori, e somma non zero se > 2 .

I giochi possono essere definiti fornendo una ontologia di ricerca con: uno stato iniziale, una funzione successore, test terminazione ed una funzione di utilità. A partire dallo stato iniziale si può costruire l'albero di gioco, da cui si può estrarre una ricerca.

In un gioco un giocatore deve applicare una strategia consistente nel gli permettere di valutare un cammino in relazione alle possibili mosse dell'altro.

Questo può essere fatto in relazione al valore di minimax di ogni nodo, ovvero all'utilità di trovarsi in quel nodo di MAX rispetto a MIN.

L'algoritmo minimax dunque non fa altro che misurare il valore di utilità corrente calcolando ricorsivamente quello dei successori, dunque ad ogni iterazione l'algoritmo esegue un SS. (completo) di cui può essere ampliato anche per più di 2 giocatori.

Essendo il numero degli stati troppo complesso da valutare bisogna applicare tecniche di ottimizzazione per rendere l'algoritmo applicabile a problemi reali: il più famoso tra essi è la potatura. Essa consiste nell'eliminare un nodo, o l'intero sottoalbero se dati due nodi $n1$ e $n2$, se $n2$ e $n1$ hanno lo stesso padre e $n2$ ha un'utilità (minimax) maggiore di $n1$, allora $n1$ può essere eliminato.

α = è il valore della scelta migliore per MAX fin qui.

β = è il valore della scelta migliore per MIN fin qui.

Grazie a questi valori possiamo potare tutti quei rami che hanno utilità peggiore di α e β .

Inoltre spesso nei giochi si presentano stati uguali (nodi), è interessante usare una tabella hash per risparmiare spazio.

Nonostante l'~~CB~~ pruning vi possono essere casi che ^{resterebbero} a cicli,
o a troppi tempi per trovare la soluzione, si usa così una funzione
di valutazione per fermare prima l'algoritmo. Ci sarà dunque una
euristica da decidere quando trasformare un nodo in foglia. Vi può
essere anche un valore di cutoff da prendere di taglio. Il
tema se secondo EVA2 non è utile.

EFFETTO ORIZZONTE quando c'è una mossa molto a vantaggio
dell'avversario ma è inevitabile, allora si prova a spingerla al di là
del valore di cutoff.

Quando c'è incertezza la complessità aumenta, bisogna valutare
esse e le possibili conseguenze. $O(b^m)$ e $O(b^{m \cdot n})$
possibilità

GAME THEORY

5/17/2018

Stefano

Analizzare giochi con mosse simultanee e parzialmente osservabili, in cui l'incertezza è data anche dagli agenti e le loro decisioni.

Può essere usata in 2 modi: AGENT DESIGN e MECHANISM DESIGN

SINGLE-MOVE GAMES (insieme contemporaneamente)

Un gioco a mossa singola è costituito da 3 componenti: ^{pag. 4} PLAYERS, ACTIONS, PAYOFF

Ogni player deve adottare una strategia, la quale può essere DET (pura), MIST (caso) o seguire un preciso piano strategico, una strategia prefissata. Data una strategia il risultato è dato da un valore numerico. Dunque una soluzione per un gioco è un profilo strategico in cui ogni giocatore sceglie.

DILEMMA DEL PRIGIONIERO.

Una strategia è dominante ^{per A} forte se per ogni azione di B la strategia è la migliore, dominante debolmente se invece B è diverso per una o non ha. Quando una strategia è dominante per tutti i giocatori si ha l'equilibrio dominante. Caso particolare è il NASH EQUILIBRIUM in cui nessun agente ha vantaggi ad uscire. ES. NUCLEARE.

Per uscire dall'equilibrio bisognerebbe comunicare, e in base a ciò si fa distinzione fra giochi coordinati e strategicamente competitivi.

REPEATED-GAMES (giochi in cui vi è la stessa scelta ripetutamente)
comportamenti tipici:

PERPETUAL PUNISHMENT: cooperare finché l'altro non tradisce

TIT-FOR-TAT = TESTIMONIA SOLO IL TUO DDOO B, MA SNETE ALTRA

ALLD / TESTER se coopera, se B non TIT

TIT-FOR-TAT / DDOO - se TIT ha a volte a volte

Vi sono delle regole per cooperare al meglio: non essere geloso, gentile, non provare rancore, punire appropriatamente.

SEQ. GAMES.

Sono i giorni ⁴⁴impegnati in cui ogni Anno differenzia dal precedente in base ai 4 anni precedenti. La migliore rappresentazione può essere fatta tramite alberi, non

- So è lo stato iniziale
- $Player(s)$ ci dice il player da fare giocare.
- $Action(s)$ la azioni possibili
- $Result(s, a)$ che ci dice la transizione al nuovo stato
- $Utility(s, p)$ che ci va a valutare l'utilità dello stato finale
- giocatore change se è un gioco stocastico.

Grazie a questa rappresentazione statale (dette EXTENSIVE FORM) possiamo
 vedere anche l'osservabilità razionale creando stati d'opinione e
 dando una soluzione qualsiasi sia le scelte degli avversari.

La LANE THEORY presenta alcune limitazioni, ovvero non funziona bene con stati e azioni continue, secondo assume che il gioco sia noto.

Vi possono essere zero vari fonti di incertezze quali:

Action: non è facile rappresentare gioco dove non si conosce

STRATEGIES: se i giocatori non sono razionali non funzionano

chances: se le possibilità non sono uniformi?

VERITIER: Non è così ^{de} sappiamo l'italiano

Per questi motivi viene usato principalmente per analizzare ambienti in equilibrio, piuttosto che agenti nell'ambiente.

MECHANISM DESIGN

ti permette di costruire sistemi intelligenti da un insieme limitato di sistemi, fondamentalmente si compone di:

- un linguaggio che descrive l'insieme delle strategie degli agenti; ^{usare} possono
- un agente distinto dal centro che raccoglie le info dagli altri player.
- regole di risultato, note al tutti che il centro utilizza per l'esito finale

ACTION (aste) è un esempio di mechanism design in cui ogni player ha sue valutazioni dei beni.

Vi sono vari tipi.

* INGLESE

Si parte da un minimo e si va a salire, vince l'ultimo che accetta.

SEALED-BID AUCTION: vince l'offerta più alta (segreta).

SEALED-BID AUCTION: " " " " " " " " ma paga la seconda

DUTCH AUCTION = vince il primo che accetta, si parte da una cifra alta.

Nelle Aste bisogna evitare collusione, altrimenti non c'è concorrenza.

* c'è il rischio di truth-revealing

LEGGERE TRAGEDIA BENI COMUNI